

Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas



FIUSAC
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



Diseño Arquitectura

PONDERACIÓN: 3 pts

 **Tiempo estimado: 10 hrs/min**

1. MARCO FORMATIVO	3
1.1. Valor	3
1.2. Competencia(s)	3
1.3. Habilidad(es) blandas a formar	3
2. Resultado del Aprendizaje	3
2.1. Objetivo SMART	3
3. Enunciado de la Práctica	4
3.1 Descripción del problema a resolver	4
3.2 Alcance de la práctica	4
4. Entregables	5
5. Material de apoyo	5
6. Recursos y herramientas a utilizar	5
7. Cronograma	6
8. Rúbrica de Calificación	6
8.1 Requisitos para optar a la calificación	6
8.2 Resumen de Puntuaciones	7
Detalle de la Calificación	8
Comentarios Generales	9

1. MARCO FORMATIVO

1.1. Valor

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Excelencia técnica	El estudiante diseña una arquitectura escalable y coherente, aplicando estándares de la industria en el modelado de microservicios y bases de datos.
Pensamiento sistémico	Al diseñar la solución, el estudiante considera la interacción entre múltiples servicios, sistemas externos y flujos de negocio complejos, evidenciando una visión integral del sistema.

1.2. Competencia(s)

Tipo de competencia	Arquitectónica y de Diseño de Software Empresarial
Competencia General	Diseñar soluciones de software escalables y mantenibles utilizando arquitecturas modernas y principios de ingeniería de software.
Competencia Específica	Diseñar una arquitectura basada en microservicios para un sistema empresarial de alta demanda.

1.3. Habilidad(es) blandas a formar

La práctica le permitirá desarrollar las siguientes habilidades:

- **Individual:** Análisis y resolución de problemas complejos, al traducir requerimientos de negocio (flujos de aprobación, integraciones externas, logging) en una arquitectura técnica coherente.
- **Individual:** Comunicación técnica escrita y visual, mediante la elaboración de diagramas de arquitectura, ER y documentos de solución que comuniquen decisiones de diseño con claridad.
- **Individual:** Toma de decisiones justificadas, al seleccionar tecnologías y patrones de diseño argumentando sus ventajas en el contexto del problema planteado.

2. Resultado del Aprendizaje

2.1. Objetivo SMART

Específico (¿Qué?)	Medible (¿Cuánto?)	Alcanzable (¿Cómo?)	Realista (¿Para qué?)	A Tiempo (¿Cuándo?)
Diseñar arquitectura de	Entregar diagrama de	Utilizando herramientas	Aplicado a un escenario real	Antes del x de Julio de 2026 a

microservicios para sistema bancario	arquitectura general, diseño de al menos 3 microservicios, diagramas ER y diagramas UML (clases y secuencia) por servicio, definición del flujo de aprobación y descripción técnica completa de la solución.	como Draw.io, Lucidchart, Miro, ArchiMate o similares, apoyándose en el módulo de autenticación desarrollado en la Práctica 2.	de alta demanda empresarial, desarrollando la competencia de diseño arquitectónico que se exige en el ejercicio profesional de la ingeniería en sistemas.	las 23:59 hrs, entregando el repositorio vía UEDI.
--------------------------------------	--	--	---	--

3. Enunciado de la Práctica.

3.1 Descripción del problema a resolver

En la institución bancaria donde el estudiante labora actualmente, el sistema de procesamiento de transacciones funciona bajo un esquema **monolítico**, lo que provoca un lento funcionamiento en épocas de alta demanda como fin de mes, pago de planillas masivas de clientes corporativos, temporadas de impuestos, entre otros. Por esta razón, se acude al equipo de trabajo para realizar el **diseño inicial de una nueva solución** que resuelva este problema.

El sistema deberá integrar los siguientes elementos:

- Un servicio de autenticación **OAuth corporativo** con tiempo de vida de token de 12 horas, complementado con el módulo de autenticación desarrollado en la Práctica 2 para gestionar permisos y accesos.
- Un **sistema core bancario / de compensación interbancaria externo** al cual el sistema envía las transacciones (transferencias, pagos, depósitos en lote) una vez aprobadas.
- Las transacciones (transferencias masivas, pagos en lote) se cargan mediante **archivos CSV**, los cuales deben ser validados por las reglas internas del negocio (saldo disponible, límites de transacción, cuentas válidas, prevención de fraude) y almacenados en un servidor (nube o FTP).
- Las transacciones siguen un **flujo de aprobación de 3 pasos (esquema maker-checker-authorizer)** por distintos usuarios antes de ser enviadas al sistema core bancario.
- Al completarse el paso 3 y aprobarse el lote de transacciones, se debe enviar un **correo electrónico a todos los clientes/beneficiarios** incluidos en ese lote informando que su transacción está en proceso.

- El sistema debe mantener un **historial consultable** de todos los lotes de transacciones procesados, incluyendo su contenido y la opción de descarga.
- Es obligatorio contar con un **sistema de logging centralizado** y auditable que registre todo lo que ocurre en el sistema.

3.2 Alcance de la práctica

El estudiante deberá entregar los siguientes elementos de diseño:

- Diagrama de Arquitectura General del sistema
- Uso e integración del servicio de autenticación de la Práctica 2
- Diseño de microservicios (mínimo 3 servicios) con separación adecuada de responsabilidades
- Diagrama ER por cada microservicio
- Diagrama de clases UML por cada microservicio (entidades principales, atributos y relaciones)
- Diagramas de secuencia UML para los flujos críticos (aprobación de transacciones de 3 pasos, envío al sistema core bancario, notificación a clientes)
- Diagrama de componentes UML mostrando la relación entre microservicios, API Gateway y sistemas externos
- Definición y documentación del flujo de aprobación de 3 pasos
- Diseño de la estrategia de almacenamiento de archivos CSV
- Diseño de la estrategia de logging centralizado
- Explicación de la comunicación entre servicios (REST y/o mensajería)
- Propuesta de API Gateway

3.3 Requerimientos técnicos

- **Arquitectura:** Basada en microservicios con API Gateway.
- **Autenticación:** Integración con OAuth corporativo (token 12h) y con el módulo de la Práctica 2.
- **Base de datos:** Una base de datos independiente por cada microservicio.
- **Almacenamiento de archivos:** Cloud Storage (AWS S3, GCP, Azure) o servidor FTP.
- **Logging:** Sistema de logging centralizado y consultable.
- **Comunicación:** REST y/o mensajería asíncrona (Kafka, RabbitMQ, NATS u otro).
- **Herramientas de modelado:** Draw.io, Lucidchart, Miro, ArchiMate, PlantUML, StarUML o similares.
- **Documentación UML:** Diagrama de clases, diagrama de secuencia y diagrama de componentes por microservicio, siguiendo notación UML estándar.
- **Docker / Kubernetes:** Opcionales, pero se valoran si se incluyen en el diseño.

4. Entregables

Tipo	Descripción
Repositorio en la nube	Repositorio privado con el código fuente de la API REST, organizado dentro de la carpeta P1 .
README en Markdown	Documento que incluye: explicación con palabras propias de los 5 principios SOLID y evidencia de su aplicación en el código (archivo/clase, justificación y fragmentos de código reales).
Enlace en UEDI	Subir el enlace del repositorio a la plataforma UEDI antes de la fecha límite.

5. Material de apoyo

- Sam Newman — *Building Microservices* (O'Reilly)
- [Documentación oficial de OAuth 2.0](#)
- [Documentación de Apache Kafka](#)
- [Documentación de RabbitMQ](#)
- [AWS Architecture Best Practices](#)
- [GCP Architecture Framework](#)
- [API Gateway Pattern — Microsoft](#)
- [Guía de diagramas con Draw.io](#)

6. Recursos y herramientas a utilizar

- Herramientas de modelado: Draw.io / Lucidchart / Miro / ArchiMate (para diagramas de arquitectura y ER).
- Repositorio: GitHub / GitLab.
- Contenedores (opcional): Docker, Kubernetes.
- Lecturas recomendadas: Building Microservices (Sam Newman), documentación oficial de las herramientas de mensajería elegidas, guías de API Gateway.

7. Cronograma

Tipo	Fecha Inicio	Fecha Fin
Asignación de Práctica	06/08/2026	06/08/2026
Elaboración	06/08/2026	13/08/2026
Calificación	15/08/2026	15/08/2026

8. Rúbrica de Calificación

8.1 Requisitos para optar a la calificación

Antes de la evaluación de la práctica, los estudiantes deben cumplir con los requisitos que se indiquen en esta sección.

Tema	Descripción	Cumple (Sí/No)
Arquitectura definida	Existe diagrama claro y coherente del sistema.	
Microservicios definidos	Separación adecuada de responsabilidades entre servicios.	
Modelo ER y UML	Diagramas ER y diagramas UML (clases, secuencia) completos por cada microservicio	
Flujo de aprobación	Flujo de 3 pasos explicado correctamente.	
Repositorio correcto	Cumple con el formato de nombre solicitado y carpeta P3 incluida.	

8.2 Resumen de Puntuaciones

Área	Puntos Totales	Puntos Obtenidos
------	----------------	------------------

1. Habilidades		
Calidad del diseño arquitectónico	20	
Modelo ER y UML correcto (por microservicio)	10	
Separación adecuada de microservicios	5	
Organización del repositorio	5	
Preguntas teóricas	10	
Sub-Total Habilidades	50	
2. Conocimiento		
Diagrama de Arquitectura	20	
Descripción de la solución (integración P2 y reglas de negocio)	10	
Reflejo de las reglas de validación del negocio	10	
Descripción técnica de tecnologías y patrones	10	
Sub-Total Conocimiento	50	
TOTAL	100	

Detalle de la Calificación

	Criterio/Nivel			Nivel de evaluación		
No.	Criterio de evaluación	Punteo maximo		Satisfactorio 100% - 61%	Necesita mejorar 60% - 0%	Punteo Obtenido
1	Habilidades	50				
1.1	Calidad del diseño arquitectónico	25	Descripción	Arquitectura coherente, escalable y correctamente segmentada, que contempla todos los componentes requeridos (OAuth, API Gateway, servicios, almacenamiento, logging).	Arquitectura incompleta, confusa o sin separación clara de componentes. No cumple con los lineamientos de microservicios.	
			Rango del punteo	20-13 pts	12-0 pts	
1.2	Modelo ER y UML correcto (por microservicio)	10		Modelo ER separado por microservicio y diagramas UML (clases y secuencia) que documentan correctamente el diseño y los flujos entre servicios.	Sin separación de bases de datos por microservicio, diseño deficiente de los modelos o ausencia de diagramas UML.	

1.3	Separación adecuada de microservicios	15		Se definen al menos 3 microservicios con responsabilidades claramente diferenciadas y sin acoplamiento indebido.	Los microservicios no están correctamente separados o sus responsabilidades se solapan.	
	Sub-Total de Puntos					
2	Conocimientos	50				
2.1	Diagrama de Arquitectura General	20	Descripción	El diagrama modela correctamente todos los componentes del sistema (servicios, API Gateway, OAuth, sistema core bancario externo, almacenamiento, logging) y su comunicación.	El diagrama es incompleto, no sigue los lineamientos de microservicios o no refleja los componentes requeridos.	
			Rango del punteo	10 - 7 pts	6 - 0 pts	
2.2	Descripción de la solución (integración con P2 y reglas de negocio)	10		La descripción detalla cómo se integra el módulo de autenticación de la Práctica 2, respondiendo a las preguntas necesarias para validar el conocimiento del diseño.	No incluye la integración con la Práctica 2 o la descripción no responde las preguntas de validación.	
2.3	Reflejo de las reglas de validación del negocio	10		La arquitectura refleja correctamente todas las reglas del negocio: flujo de aprobación de 3 pasos, envío de correos al aprobar, historial consultable y logging auditable.	El diseño omite o modela incorrectamente alguna de las reglas de negocio requeridas.	

2.4	Descripción técnica de tecnologías y patrones	10		El documento de solución describe de forma concisa las tecnologías elegidas, los patrones de diseño a implementar y justifica su uso en el contexto de la arquitectura propuesta.	La descripción es vaga, sin justificación del uso de tecnologías o sin mención de patrones de diseño.	
	Sub-Total de Puntos	50				
	Total	100				